

Ch. 4 - File System

I requisiti essenziali per la memorizzazione **a lungo termine** sono:

- Capacità di *salvare grandi quantità di informazioni*;
- *Persistenza delle informazioni* oltre la vita del processo che le utilizza;
- *Accessibilità* delle info *da più processi* simultaneamente.

Il **File System** offre una soluzione di gestione delle informazioni, che vengono memorizzate tramite l'utilizzo di dischi magnetici o SSD e le operazioni su di esse. Il FS offre un'*astrazione del file*, che risolve il problema di memorizzazione consentendo la *persistenza*, l'accesso multiplo e la gestione di grandi volumi di dati.

Quindi i File System sono un metodo per *organizzare e memorizzare informazioni*. Sono organizzati in **file e directory**.

File

I File fungono da metodo di *astrazione per salvare e leggere informazioni* su disco nascondendo i dettagli implementativi.

I file vengono identificati tramite **nomi**, che possono variare in base al SO, ed **estensioni**, che tipicamente hanno un significato specifico in base ai programmi che li utilizzano. I file possono avere due diverse strutture:

1. **Sequenza non strutturata di byte**: Il significato dei dati è determinato dai programmi a livello utente e non dal SO
2. **Sequenza di record di lunghezza fissa**: Letture e scritture avvengono a unità di record.
3. **File come albero di record**: Con lunghezze variabili e capacità di ricerche rapide basate su chiavi specifiche. Utilizzati in sistemi mainframe.

E possono dividersi in diversi tipi:

- **File e Directory Normali**:
 - *SO*: Utilizzati in UNIX e Windows;
 - *File Normali*: Contengono info utente e sono la forma più comune;
 - *Directory*: File di sistema per mantenere la struttura del FS
- **File Speciali**:

- *File Speciali a caratteri*: Usati per modellare dispositivi I/O come terminali e stampanti;
- *File Speciali a blocchi*: Usati per modellare dischi.

Tipi di file normali

- *File ASCII*: Composti da righe di testo visualizzabili e stampabili;
- *File binari*: Non leggibili come testo, hanno una struttura interna conosciuta dai programmi che li utilizzano, come file eseguibili e archivi.

Accesso ai file:

- *Accesso sequenziale*: All'inizio unico metodo disponibile.
- *Accesso casuale*: Permette la lettura di byte o record in posizione diverse senza seguire una sequenza.

L'accesso *Casuale* è fondamentale per applicazioni come sistemi di database. I SO inoltre associano ai file ulteriori informazioni dette *attributi* o metadati, che sono cruciali per la protezione o la gestione del file stesso.

Operazione sui File

- `create` Crea un file senza dati
- `delete` Elimina un file per liberare spazio su disco
- `open` Apre il file per consentire al sistema di caricare in memoria attributi ed indirizzi del disco
- `close` Chiude il file al termine degli accessi per liberare spazio nelle tabelle interne
- `read` Legge i dati di un file
- `write` Scrive dati nel file
- `append` Aggiunge dati alla fine del file
- `seek` Riposiziona il puntatore del file su una posizione specifica, permettendo la lettura/scrittura da quella posizione
- `getAttributes` Legge gli attributi di un file
- `setAttributes` Modifica gli attributi di un file da parte dell'utente
- `rename` Rinomina il file.

Es. Aprire un file e scrivere:

```

1
1  int fd = open("foo.txt", O_WRONLY | O_CREAT |
2  O_TRUNC);
3  char buf[] = "Hi there!";
4  write(fd, buf, strlen(buf));
   close(fd);

```

Dove:

- `O_CREAT` crea il file se non esiste;
- `O_WRONLY` apre il file in modalità scrittura;
- `O_TRUNC` se il file esiste ne tronca il contenuto a dimensione 0.

Sovrascriverà quindi il contenuto di `foo.txt` con "Hi there!", o creerà un nuovo file con questo contenuto se `foo.txt` non esiste.

Directory

Per **directory** si intendono file che tengono traccia degli altri file all'interno del File System. Storicamente si utilizzavano *sistemi di directory a livello singolo* (una directory 'root' che contiene tutti i file). Altrimenti, vengono implementati nei FS *sistemi di directory gerarchici*, dove:

- I file sono organizzati in gruppi correlati mediante directory ramificate
- Struttura ad albero per separare ed organizzare logicamente i file
- Ogni utente può avere una directory principale privata in ambienti condivisi
Nasce la necessità quindi di definire i percorsi dei file, tramite i **nomi**:
- *Nomi di percorso Assoluti*: partono dalla directory principali e conducono al file
- *Nomi di percorso Relativi*: partono dalla directory di lavoro dell'utente

Operazioni sulle Directory

- `create` Crea una directory vuota
- `delete` Elimina una directory, possibile solo se vuota
- `opendir` Apre una directory per leggerne il contenuto
- `closedir` Chiude la directory dopo la lettura
- `readdir` Restituisce la prossima voce in una directory aperta senza esporre la struttura interna

- `rename` Rinomina la directory

Gestione dei link ed accessi avanzati

- `link` crea un 'hard link' collegando un file esistente ad un nuovo percorso, condividendone l'i-node
 - `unlink` rimuove una voce di directory, cancellando il file se è l'unico link
- Link simbolici*: Variante degli 'hard link', possono puntare a file locati su dischi o computer diversi.

Implementazione del File System

Un disco può essere suddiviso in *più partizioni*, ciascuna con il proprio FS indipendente, ed i metodi di strutturazione del FS variano a seconda dell'epoca del computer, influenzando come i dati vengono gestiti e acceduti.

Master Boot Record - vecchio stile

- MBR nel BIOS, situato nel settore 0 del disco, è essenziale per l'avvio del pc.
- Contiene la tabella delle partizioni con i dettagli su inizio e fine di ciascuna partizione, ed identifica la partizione attiva da cui avviare il sistema.

All'avvio:

- Il BIOS legge MBR per trovare la partizione attiva
- Carica il *boot block* della partizione attiva per avviare il SO

Ogni partizione inizia con un boot block seguito da vari elementi di sistema.

Unified Extensible Firmware Interface

UEFI sostituisce il vecchio BIOS tradizionale.

- Avvio più veloce, migliore compatibilità
- Interfaccia utente avanzata
- Sicurezza

Inoltre, supporta i dischi moderni con GPT:

GUID Partition Table: Sistema avanzato di gestione delle partizioni

- Supporta dischi fino a 8ZiB e consente un numero illimitato di partizioni
- Include *backup della tabella delle partizioni* per maggiore sicurezza
- Utilizza un controllo di integrità (CRC) per prevenire corruzione dei dati.

EFI System Partition: Partizione speciale sui dischi GPT

- Archivia i file di avvio come bootloader, driver e utility di diagnostica

- Essenziale per avviare il SO.

Secure Boot: Funzionalità UEFI per impedire l'avvio di SW non autorizzato.

- Controlla le *firme digitali* di bootloader, driver e SO
- Avvia solo SW autorizzato e firmato, bloccando malware e rootkit.

Protegge contro attacchi all'avvio, mantiene l'integrità del SO ed aumenta la sicurezza per utenti domestici e aziende.

File nei File System

L'*obiettivo principale* è la gestione dell'associazione tra i file ed i blocchi del disco su cui sono memorizzati. È fondamentale per assicurare l'integrità, l'accesso efficiente e la gestione dello spazio su disco.

Allocazione contigua:

- File memorizzati come *sequenze contigue di blocchi sul disco*.
- Semplice da implementare e dispone di un'alta efficienza di lettura.
Tuttavia, col passare del tempo *i dischi si frammentano* a causa della rimozione di file, e ne sussegue un *problema di allocazione di nuovi file in spazi liberi* frammentati.

Allocazione a liste concatenate:

- File organizzati come *liste concatenate di blocchi su disco*, in cui *ogni blocco contiene una parte di dati ed un puntatore al blocco successivo*.
- Efficiente utilizzo di tutti i blocchi disponibili sul disco, e minima frammentazione interna.
- Ogni voce di directory traccia solo l'indirizzo del primo blocco di un file
L'accesso casuale ai dati però è estremamente lento, ed ogni blocco ha una dimensione effettiva ridotta a causa dello spazio occupato dal puntatore.

Allocazione a liste concatenate con FAT

Ottimizzazione dell'allocazione a liste concatenate che sposta i puntatori in una tabella di memoria (*FAT - File Allocation Table*) in RAM.

La FAT deve essere mantenuta *interamente* in memoria, e ne richiede una quantità significativa. Lo spazio e la velocità influenzano la dimensione della voce della FAT, e non risulta ottimale.

I-Node - Index Node

Si tratta di una struttura dati che contiene tutte le informazioni su un file, esclusi

nome e contenuto. Include metadati come permessi, proprietario, timestamp ed indirizzi dei blocchi di dati.

Ogni file e directory è rappresentato da un I-Node univoco, indicizzato in una tabella di I-Node. Gli I-Node separano le informazioni sul file dalla sua posizione fisica sul disco, offrono una gestione più dettagliata dei metadati (inclusi permessi e proprietà) e tendono ad essere più efficienti e performanti della FAT, specialmente su dischi di grandi dimensioni.

Directory Nei File System

Le directory mappano i nomi ASCII dei file sulle informazioni necessarie per localizzare i dati su disco. I *metodi di allocazione* variano a seconda del SO, includendo indirizzi di blocchi contigui, il primo blocco nelle liste concatenate, o i numeri degli I-Node. Nei moderni sistemi i nomi dei file possono variare con caratteri da 1 a 255. Per gestire questa variabilità, si utilizzano due modi per strutturare le directory:

- *Struttura con Header di lunghezza fissa* : ogni voce nella directory inizia con un header di lunghezza fissa e termina con il nome del file. Ogni file termina con un carattere speciale, che viene utilizzato anche più volte per "riempire" (padding) il nome affinché questo abbia un numero intero di parole (32 o 63 bit in base al sistema). Questo sistema può tuttavia portare a una Frammentazione Interna, in quanto quando un file viene cancellato il suo spazio potrebbe non poter essere utilizzato in modo efficiente se il nuovo file da inserire non ha la stessa lunghezza (si possono creare piccoli "buchi" inutilizzabili).
- *Utilizzo di Heap per i nomi dei file* : i nomi dei file sono salvati in un heap, struttura dati che permette di memorizzarli in modo dinamico e non sequenziale. Quando si aggiunge il nome di un file, lo spazio viene allocato dinamicamente dall'heap, e non è necessario riempire i nomi dei file fino a raggiungere lunghezza fissa.

In aggiunta ai due metodi principali, per implementare la ricerca dei file all'interno di una directory, è stato introdotto l'uso delle *tabelle hash*. Quando si cerca un file, il suo nome viene *trasformato* attraverso una funzione di hashing in un indice numerico che punta ad una posizione specifica della tabella. Se più file dovessero generare lo stesso indice (collisione) allora viene utilizzata una lista concatenata per collegare i file che hanno generato lo stesso hash.

Un'ulteriore ottimizzazione è l'uso di **caching** per memorizzare i risultati delle ricerche precedenti.

File condivisi e Link

I **File condivisi** sono essenziali in ambienti collaborativi per permettere a più utenti di lavorare sugli stessi file. Tipi di link:

- **Hard Link**: Puntano all'I-Node di un file condiviso
- **Soft Link**: Puntano al nome di un file
Un file con hard link viene *rimosso solo quando non ci sono più riferimenti ad esso*.
- Gli Hard Link sono vantaggiosi poiché utilizzano un solo I-Node indipendentemente dal numero di link, ma allo stesso tempo il file permane fino all'eliminazione degli hard link, potenzialmente creando confusione sulla proprietà del file
- I Soft Link sono più flessibili, poiché possono riferirsi a nomi di file oltre i confini del file system e su macchine remote, ma sono meno efficienti in termini di spazio, e diventano invalidi alla rimozione del file originale

Spazio su disco

I file sono generalmente memorizzati sul disco, e per farlo si usa:

- Allocazione contigua
- Suddivisione in blocchi non contigui.
L'allocazione contigua richiede spostamenti di file se le loro dimensioni aumentano, mentre i blocchi non contigui spezzettano i file in blocchi di dimensioni fisse, consentendo una maggiore flessibilità ed un migliore utilizzo dello spazio su disco.
La scelta della dimensione dei blocchi dipende da una considerazione dell'ottimo per bilanciare il tempo di trasferimento e l'efficienza dello spazio:
- Blocchi più grandi consentono di trasferire più dati in una singola operazione di lettura o scrittura, tuttavia portano ad uno spreco di spazio se i file sono piccoli.
- Blocchi piccoli invece riducono lo spreco di spazio poiché è meno probabile che rimangano aree non utilizzate all'interno di un blocco. Tuttavia gestire file su più blocchi può aumentare il tempo necessario per accedere ai dati.

Per tenere traccia dei blocchi liberi su disco si utilizzano:

1. Lista concatenata:

- Nella lista ci sono i blocchi liberi, dove ogni blocco contiene numeri di blocchi del disco liberi. Richiede meno spazio solo se il disco è quasi pieno.

2. BitMap:

- Un bit per ogni blocco del disco (1 se libero, 0 se già allocato). Richiede meno spazio della lista concatenata, tranne in dischi quasi pieni

Quote del disco : Limitazioni gestite dal SO sull'utilizzo dello spazio del disco, imposte sui singoli utenti per evitare l'utilizzo eccessivo di spazio. Due tipi:

- **Limite Soft**: il limite può essere momentaneamente superato, ma non permanentemente. Se infatti lo spazio utilizzato rimane sopra il limite soft per un certo periodo di tempo (determinato dal sistema) l'utente non sarà in grado di salvare nuovi dati finché non libera spazio o ottiene una nuova quota.
- **Limite Hard**: il limite non può essere superato in nessuna circostanza.

Performance

Velocità di accesso:

- **Memoria**: Accesso ultraveloce
- **Disco magnetico**: Più lento a causa del tempo di ricerca della traccia
Progettando dei FS con diverse ottimizzazioni per migliorare le prestazioni, considerando le significative differenze nel tempo di accesso e riducendo al minimo il numero di accessi a disco, il tempo di ricerca e l'utilizzo dello spazio, si ottiene una notevole ottimizzazione nel FS.
- **Buffer Cache**: Utilizzata per ridurre i tempi di accesso al disco, mantenendo i blocchi più usati in memoria
- **Allocazione dei blocchi e Read Ahead**: Tecniche di allocazione intelligente per le quali blocchi vicini allocati nello stesso cilindro per minimizzare il movimento del braccio del disco, e bitmap in memoria per allocare blocchi adiacenti e migliorare l'efficienza di scrittura sequenziale.
- **Deframmentazione**: Riorganizza i file per essere contigui e raggruppa lo spazio libero.

Concetti di *Caching*:

- *Buffer Cache*: Memorizza i blocchi del disco in RAM per ridurre gli accessi al disco.
- *Page Cache*: Memorizza le pagine del FS virtuale in RAM prima di passare al driver del dispositivo.

Per implementare la cache si utilizzano algoritmi che controllano se un blocco necessario è già presente in cache prima di effettuare un accesso al disco. L'algoritmo *LRU* (Least Recently Used) è il più comune, ma può causare problemi di incoerenza se non gestito correttamente, specialmente con blocchi critici come gli I-node. Alcuni SO dividono i blocchi in categorie basate sull'importanza per evitare problemi d'incosistenza.

- *Compressione*: il processo di ridurre la dimensione di un file eliminando ridondanze o utilizzando metodi di codifica efficienti. Questo permette di risparmiare spazio di archiviazione e di velocizzare il trasferimento dei dati.
- *Deduplicazione*: utile per eliminare i duplicati di file, può essere eseguita in tempo reale o come processo post-elaborazione, a seconda delle esigenze del sistema e dell'hardware disponibile.
- *Posizionamento degli I-node* : la posizione degli I-node influisce significativamente sulle prestazioni del file system:
 - *Posizionamento tradizionale* : posizionati vicino all'inizio del disco, porta a tempi di ricerca più lunghi, poiché il braccio del disco deve spostarsi dall'inizio del disco ai blocchi dati sparsi.
 - *Centrare gli I-node* : Un'alternativa più efficiente è quella di posizionare gli I-node al centro del disco per ridurre il tempo di ricerca medio. Inoltre si può dividere il disco in gruppi di cilindri, ciascuno con i proprio I-node, blocchi e lista dei blocchi liberi.

Affidabilità

Fondamentale per garantire la protezione dei dati. Le principali minacce includono *guasti del disco*, *interruzioni di energia*, *bug del software*, *errori umani*, *perdite o furti* e *malware*. Nasce la necessità del *backup*, per salvaguardare informazioni importanti come documenti o database.

- *Backup completo*: Copia totale dei dati;
- *Backup incrementale*: Copia dei soli file modificati dall'ultimo backup completo, riducendo tempo e spazio richiesti.

Tipologie di BackUp:

- *Backup Fisico*: Copia sequenziale di tutti i blocchi del disco
- *Backup Logico*: Selezione e copia di soli file e directory specifici, ignorando file di sistema e blocchi danneggiati.

Inoltre:

- *Comprimere i dati* riduce lo spazio necessario ma aumenta il rischio di perdita di dati a causa di errori di compressione
- *Backup di file system attivi* richiede l'utilizzo di snapshot per garantire coerenza durante il backup di un sistema in uso
- *Sicurezza dei backup* incide per prevenire perdite o danni

Backup Fisico

Considerazioni:

- *Efficienza*: Semplice e veloce, eseguito alla velocità del disco
- *Gestione dei blocchi danneggiati*: Necessità di evitare blocchi danneggiati per prevenire errori di lettura
- *File non necessari*: Necessità di evitare la copia di file di sistema come paginazione o ibernazione.

Manca tuttavia di flessibilità, poiché è difficile saltare directory specifiche o fare backup incrementali, e non è possibile ripristinare file individuali senza un intero ripristino del sistema.

Backup Logico

Funzionamento:

- Parte dalle *directory specifiche* ed effettua il backup di tutti i file e directory modificati a partire da una data specifica.
 - Ideale per backup incrementali
- *Recupero facilitato*: Consente il ripristino semplice di file o directory specifici grazie alla precisa identificazione dei dati salvati
- *Algo di backup in UNIX*:
 - Include file e directory modificati e tutte le directory lungo il percorso verso i file modificati.

rsync

`rsync` è un comando utilizzato nei sistemi UNIX per la *sincronizzazione di file/cartelle tra due location differenti*. Ottimizza il trasferimento dati *trasmettendo solo le parti di file modificate*. Risulta ideale per backup, ripristino e sincronizzazione di dati in ambienti di rete.

Coerenza

Fondamentale per *mantenere l'integrità dei dati*. Problemi di incoerenza possono sorgere a seguito di *crash* durante la scrittura dei blocchi. Tuttavia esistono degli strumenti di *utility* (`fsck` per UNIX o `sfc` per Windows) per verificare la coerenza, eseguite all'avvio, specialmente dopo un crash.

File System con Journaling

Il **Journaling** *registra anticipatamente le operazioni da eseguire in un log per garantire la coerenza in caso di crash*. Nello specifico, il **journal** è un registro che *tiene traccia delle modifiche che verranno apportate al file system prima che esse avvengano effettivamente*.

1. **Fase di registrazione:** Prima di eseguire qualsiasi modifica, il FS scrive un record nel journal. Questo record *descrive l'operazione che verrà eseguita*.
2. **Fase di esecuzione:** Il FS procede con la modifica effettiva dei dati su disco
3. **Fase di conferma:** Una volta completata l'operazione, il FS aggiorna il journal per indicare che l'azione è stata completata con successo.

Se si verifica un crash prima che una modifica sia completata, *al riavvio successivo il FS consulta il journal, che se trova operazioni registrate ma non confermate procede a completarle*.

Il Journaling favorisce l'*integrità dei dati* riducendo la possibilità di corruzione del FS ed il *recupero rapido* dopo un crash.

Eliminazione sicura:

- La cancellazione standard non rimuove fisicamente i dati dal disco, lasciandoli vulnerabili agli attacchi
- L'*eliminazione sicura* richiede la distruzione fisica o la sovrascrittura approfondita dei dati

Dati Residui:

- È consigliato inserire sequenze di 0 e numeri casuali, ripetendo l'operazione almeno 3-7 volte.

Cifratura del disco:

- La soluzione più efficace per proteggere i dati è cifrare l'intero disco con algoritmi robusti come l'AES

File System Virtuali

Virtual File System (VFS):

- Struttura che *permette di integrare vari file system in una struttura unificata*
- Si basa su un livello di codice comune che interagisce con i FS reali sottostanti

Interfacce:

- *Interfaccia superiore*: Interagisce con le chiamate di sistema POSIX dei processi utente
- *Interfaccia inferiore*: Composta da decine di funzioni che il VFS può inviare ai FS sottostanti

Funzionamento e struttura

Superblock: Descrittore di alto livello di un file system specifico nel VFS:

- Informazioni cruciali sul FS, come tipo o dimensione
- Usato per identificare ed interagire con il FS sottostante

V-Node: Astrazione di un file individuale nel VFS, rappresentandone un nodo

- *Contiene metadati* come permessi, proprietà, dimensione del file e riferimenti ai dati effettivi su disco
- Sfruttati per *fornire un accesso indipendente dal file system ai file*, permettendo operazioni di lettura, scrittura e gestione dei file.

Directory: Struttura che gestisce organizzazione e mapping dei file e delle sottodirectory nei VFS

- *Permette al VFS di mappare i nomi dei file ai loro V-Node* corrispondenti
- Facilita navigazione ed accesso ai file, consentendo agli utenti e ai processi di interagire con un'interfaccia unificata

Registrazione FS con il VFS: I File System forniscono un vettore di funzioni richieste dal VFS al momento della registrazione

- Permette al VFS di sapere come eseguire specifiche operazioni su un FS registrato

Montaggio e uso del FS: Al *montaggio* il FS fornisce informazioni al VFS (es.

superblock)

Gestione delle richieste I/O: Tracciamento dei file aperti nei processi utente tramite V-Node e tabelle dei descrittori dei file.

- Chiamate come `read` seguono il puntatore della tabella dei descrittori ai V-Node e alle funzioni del FS reale

Aggiunta di nuovi FS: Progettisti devono fornire funzioni che rispettino l'interfaccia VFS, che rende possibile la gestione trasparente di FS eterogenei.

Raid

Redundant Array of Inexpensive/Independent Disk, tecnologia che migliora le prestazioni e l'affidabilità della memoria non volatile attraverso la gestione di dischi multipli. Ne esistono di diversi tipi:

- **RAID di Livello 0** : Utilizza lo *striping*, ossia i dati sono divisi in segmenti più piccoli (stripes) e distribuite simultaneamente su più dischi del RAID. Ciò migliora le prestazioni in quanto i dati possono essere letti e scritti in parallelo. Tuttavia Raid 0 non fornisce alcuna tolleranza agli errori o ridondanza, per cui se un singolo disco fallisce tutti i dati vengono persi.
- **RAID di livello 1** : Utilizza il *mirroring*, per cui i dati vengono duplicati su due dischi separati. Ciò offre ridondanza, che migliora la tolleranza agli errori poiché i dati sono mantenuti su copie identiche in entrambi i dischi. Se un disco fallisce, l'altro può continuare senza perdita di dati.
- **RAID di livello 2** : Utilizza la *codifica di Hamming* per la correzione degli errori e lo *striping* a livello di bit. Questo tipo di RAID è molto complesso e richiede un gran numero di dischi, motivo per cui è raramente implementato in pratica.
- **RAID di livello 3** : Utilizza lo *striping a livello di byte* con un *singolo disco dedicato alla parità*, che consente di ricostruire i dati in caso di guasto di un disco. Le unità devono essere sincronizzate poiché i dati sono distribuiti a livello di byte.
- **RAID di livello 4** : Simile al RAID 3 ma utilizza lo *striping a livello di blocco* anziché a livello di byte. Un singolo disco è dedicato alla parità, il che può creare un collo di bottiglia durante le operazioni di scrittura poiché tutte le operazioni di parità devono essere scritte su un unico disco.
- **RAID di livello 5** : Distribuisce i dati e la parità tra tutti i dischi, eliminando il collo di bottiglia presente in RAID 4. E' uno dei livelli RAID più utilizzati poiché offre un buon equilibrio tra prestazioni, capacità e tolleranza agli errori.

- **RAID di livello 6** : Simile a RAID 5 ma utilizza *due blocchi di parità distribuiti*, il che consente di tollerare la perdita di due dischi anziché uno solo. Questo migliora ulteriormente l'affidabilità, ma a costo di una maggiore complessità e overhead.
- **RAID 0+1** : Combina lo *striping* di RAID 0 con il *mirroring* di RAID 1. Questo offre sia le prestazioni di RAID 0 che la ridondanza di RAID 1, ma richiede un minimo di quattro dischi.